Compilers

Guillermo A. Pérez

Lecture 2: LLVM IR, Specifications and Examples

TL;DR: This lecture in short

What is LLVM? IR?

LLVM is the most widely used compiler framework. It is easier to understand and re-use than previous compilers. It's intermediate representation is an assembly-like language with infinitely many registers.

Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **LLVM IR and Transform Pipeline.** David Chisnall. 2017. Slides from LLVM Summer School at Paris.
- Online: Wikipedia, LLVM, ANTLR, ...

Required and target competences

What tools do we need?

Familiarity with assembler (computer systems and architecture)

What skills will we obtain?

- theory: SSA IR, control flow graph
- practice: how to code in LLVM IR

How will these skills be useful?

- It will allow you to build your compiler with LLVM IR as target language.

Outline

1. LLVM IR overview
2. Static single assignment
3. Translating to SSA (for the lazy)
4. Basic blocks and control flow in LLVM IR
5. Translating to SSA (the hard way)
6. Programming with LLVM
7. LLVM example: SLP compiler

What is LLVM IR?

LLVM IR

- Unlimited single-assignment register machine instruction set
- Strongly typed
- Has three common representations:
  - Human-readable assembly (.ll files)
  - Dense "bitcode" assembly (.bc files)
  - C++ classes

How is it different from assembly?

Unlimited registers?

- Real CPUs have a fixed number of registers
- LLVM IR has an infinite number of them
- New registers are created to hold the result of every instruction (remember SLPs?)
- When translating LLVM IR to real assembly
  - The code generator's register allocator maps the infinite set of registers to the finite physical set
  - The type legalization maps LLVM types to machine types

Static single assignment

SSA

- Every register can be assigned at most once
- Most imperative languages allow variables to be variable
- SSA registers are not variables
- SSA makes the flow of data explicit!

Advantages

It simplifies and improves the results of several optimizations by simplifying the properties of variables/registers:

- Registers correspond to a single value.
- Instruction results go to output registers, and stay there.

Example: unused values (1/2)

```
1   x = 1;
2   x = 2;
3   y = x;
```

Useless assignment

It is obvious for humans that in the above code, the first assignment to x is useless! Also, y is being assigned the constant 2.

Hard algorithms

If you want to code a program to do that automatically, you would have to find all uses of x reachable without re-assigning it...

Example: unused values (2/2)

```
1   x1 = 1;
2   x2 = 2;
3   y = x2;
```

Easy algorithms

If the code is in SSA, as above, then it is easy to write an algorithm which removes all registers/variables that are assigned but never used! A second algorithm could replace constant registers by the actual constants.

Translating to LLVM IR

```
1  int a = foo();
2  a++;
```

Translating to LLVM IR

```
1  int a = foo();
2  a++;
```

```
1  %a = call i32 @foo()
2  %a = add i32 %a, 1
```

Translating to LLVM IR

```
1  int a = foo();
2  a++;
```

```
1  %a = call i32 @foo()
2  %a = add i32 %a, 1
```

Nope!

The interpreter (or compiler) will complain about a being assigned twice.

- So the front end should keep track of which registers hold values returned by functions at any point in the code!
- How do we keep track of new values?

Correct LLVM IR the easy/lazy way

```
1  ; int a
2  %a = alloca i32, align 4
3  ; a = foo()
4  %0 = call i32 @foo()
5  store i32 %0, i32* %a
6  ; a++
7  %1 = load i32* %a
8  %2 = add i32 %1, 1
9  store i32 %0, i32* %a
```

Lazy solution: generate temporal registers automatically

- Each expression is translated and assigned to a temporal register
- Assignments to variables become assignments to dynamic memory (which is not SSA in LLVM)

Is it too lazy?

It is quite bad

- Loooots of redundant memory operations
- Stores followed immediately by loads

However...

LLVM IR optimization passes can clean it up. For instance:

- Scalar Replacement of Aggregates (SROA) — as long as the `alloca` is in the entry block
- mem2reg

Basic blocks and terminators

Definition (Basic block)

In LLVM IR, a basic block is a sequence of instructions that is executed in order. Basic blocks have to end with a terminator.

Definition (Terminator)

A terminator is an intraprocedural flow-control instruction.

- `call` is not a terminator because execution resumes from the next instruction after the call
- `invoke` is indeed a terminator because execution can stop and continue from an exception-handling block

Branches and control flow

Branches in assembly

In assembly languages, jumps or branches are how the control flow of a program is managed. Not only intra-procedural control flow, but also amongst procedures and functions.

Branches in LLVM IR

LLVM IR has both conditional and unconditional jumps/branches. They are terminators and can only target blocks within the same procedure.

Branch targets

Basic blocks are branch targets in LLVM IR. However, jumps can only go the beginning of a basic block (by definition of basic block).

Example: branches for conditionals

```
1         ; set %tmp iff %a > %b
2         %tmp = icmp sgt i32 %a, %b
3         ; goto iftru if $tmp is set
4         ; goto iffls otherwise
5         br i1 %tmp, label %iftru, label %iffls
6 iftru: %c = 1
7         ; goto end
8         br label %end
9 iffls: %c = 2
10        br label %end
11 end:
```

Example: branches for conditionals

```
1         ; set %tmp iff %a > %b
2         %tmp = icmp sgt i32 %a, %b
3         ; goto iftru if $tmp is set
4         ; goto iffls otherwise
5         br i1 %tmp, label %iftru, label %iffls
6  iftru: %c = 1
7         ; goto end
8         br label %end
9  iffls: %c = 2
10        br label %end
11 end:
```

Nope! Will not compile because:

- The assignment `%c = 1` is not valid LLVM IR
- We cannot assign twice to the same register!

The control flow graph

Definition (CFG)

It is a graph-based representation of all the paths that might be traversed through a program during its execution.

- Each node in the CFG corresponds to a basic block.
- Directed edges represent jumps/branches in control flow from the end of the source block to the beginning/entry of the target block.

Exercise

What is the CFG of the preceding not valid LLVM IR code?

C++ CFG exercise

Exercise

Draw the CFG of the following snippet.

```
1  for (vector<aiger_symbol*>::iterator k =
2    latches.begin(); k != latches.end(); k++) {
3      unsigned var = *j;
4      if (var == 1) {
5          swr = swr.Cofactor(
6                mgr->bddVar((*k)->lit));
7      } else if (var == 0) {
8          swr = swr.Cofactor(
9                ~mgr->bddVar((*k)->lit));
10     } else {
11         BDD var_bdd = mgr->bddVar(*j);
12     }
13     j++; l++;
```

How do we fix this?

```
1         %tmp = icmp sgt i32 %a, %b
2         br i1 %tmp, label %iftru, label %iffls
3 iftru:  %c = 1
4         br label %end
5 iffls:  %c = 2
6         br label %end
7 end:
8         %d = %c
```

Steps

1. Try to SSA it: rename c to c_1 and c_2, in each branch
2. But then ... what is d going to be assigned?!

The Phi function

What is it?

Mathematically $d = \Phi(c_1, c_2)$ assigns to d the value of c_1 if it is defined and the value of c_2 otherwise.

```
1   %d = phi i32 [%c1, %iftru], [%c2, %iffls]
```

How do we use it in LLVM IR?

In LLVM IR, Φ is an instruction used to select a value depending on the predecessor of the current block while executing the program.

One last step

Assigning constants

There is no LLVM IR instruction to assign a constant to a register! So the actual Φ instruction we need to use is the following.

```
1   %d = phi i32 [1, %iftru], [2, %iffls]
```

We can do better!

Using a select instruction we get the following code without jumps.

```
1   %d = select i1 %tmp, i32 1, i32 2
```

LLVM IR: further questions

Can we minimize the usage of Phi?

Yes! Using the notion of node dominance and dominance frontier we can characterize when Φ instructions are needed. In the CFG:

- A node v strictly dominates a node u if any path going to u has to go through v. (Non-strictness defined if equality is allowed.)
- A node w is in the dominance frontier of v if v does not strictly dominate w but it does dominate some immediate predecessor u of w.
- Nodes in the dominance frontier are the only ones which might need Phi.

LLVM IR: further questions

Can we minimize the number of used registers?

Going from LLVM IR to real assembly, we need to fit everything into finitely many registers. In compiler optimization, this is called register allocation. We will come back to the following later:

- Chaitin and NP-completeness of optimal allocation
- Strahler number
- Sethi-Ullman algorithm